



Open in app

Get started



Published in Towards Data Science



Yohan Jeong

Follow

Apr 21, 2021 · 6 min read · Listen



Save



Item-Based Collaborative Filtering in Python

The practice of making the item-based collaborative filtering in python.



Photo by [CardMapr.nl](#) on [Unsplash](#)

Item-based collaborative filtering is the recommendation system to use the similarity between items using the ratings by users. In this article, I explain its basic concept and practice how to make the item-based collaborative filtering using



[Open in app](#)[Get started](#)

similar movies. Consider the following table for movie ratings.

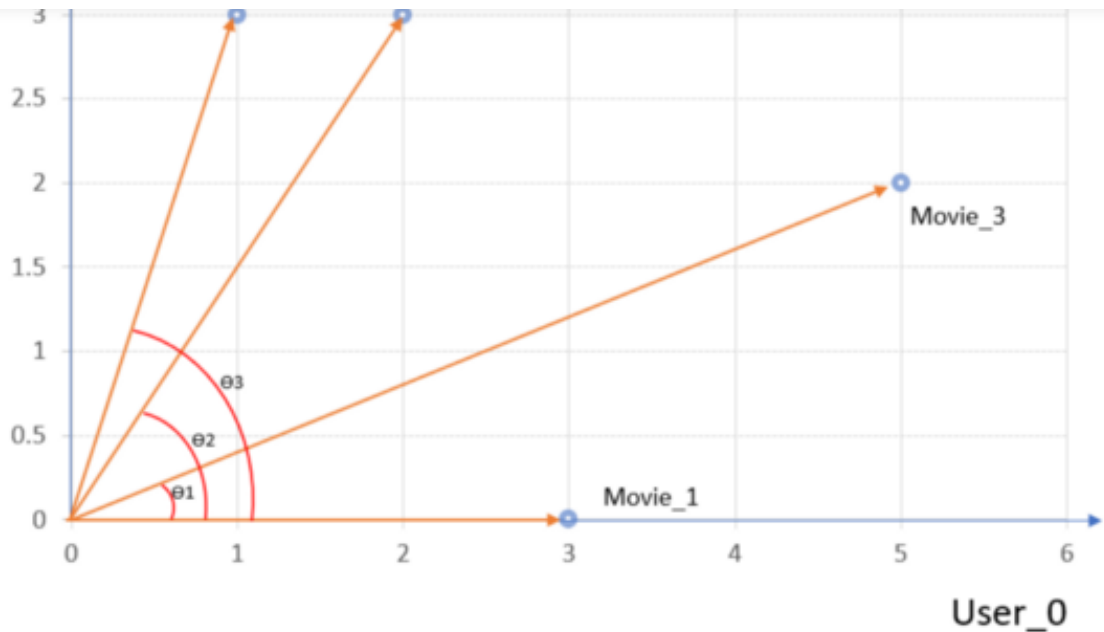
	User_0	User_1
Movie_0	2	3
Movie_1	3	?
Movie_2	1	3
Movie_3	5	2

In this example, the rating for `Movie_1` by `user_1` is empty. Let's predict this rating using the item-based collaborative filtering.

- **Step 1: Find the most similar (the nearest) movies to the movie for which you want to predict the rating.**

There are multiple ways to find the nearest movies. Here, I use the cosine similarity. In using the cosine similarity, replace the missing value for 0. Look at the graph below. The *x-axis* represents ratings by `user_0` and the *y-axis* ratings by `user_1`. Then, we can find points for each movie in the space. For example, `Movie_3` corresponds to the point (5,2) in the space.



[Open in app](#)[Get started](#)

The cosine similarity uses $\cos(\theta)$ to measure the distance between two vectors. As θ increases, $\cos(\theta)$ decreases ($\cos(\theta) = 1$ when $\theta = 0$ and $\cos(\theta) = 0$ when $\theta = 90$). Therefore, as the value of θ is smaller, the two vectors are considered closer (the similarity gets greater). Since θ_1 is the smallest and θ_3 is the largest, **Movie_3** is the closest to **Movie_1**, and **Movie_2** is the farthest.

What is noteworthy here is that the similarities between movies are determined by **ALL USERS**. For example, the similarity between **Movie_1** and **Movie_3** is calculated using the ratings by both of **User_0** and **User_1**.

- **Step 2: Calculate the weighted average of the ratings for the most similar movies by the user.**

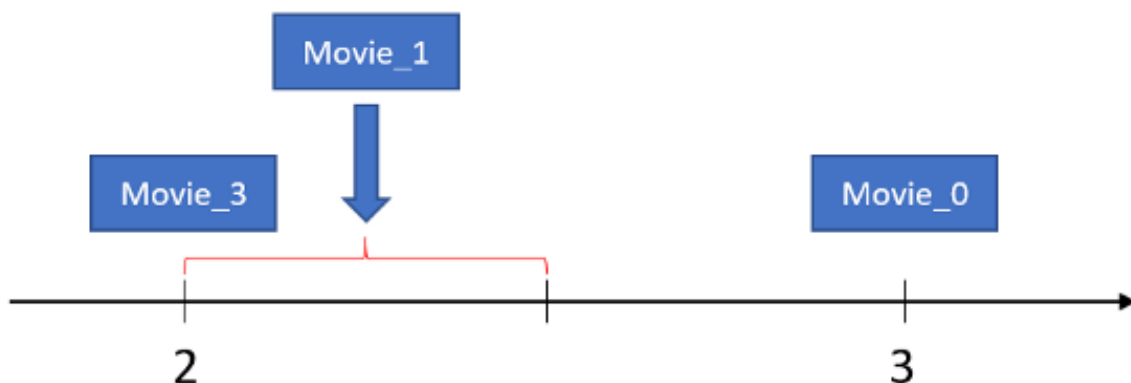
A user gives similar ratings to similar movies. Therefore, when we predict a rating for a movie by a user, it is reasonable to use the average of ratings for the similar movies by the user. Set the number of the nearest neighbors as 2. Then we use **Movie_3** and **Movie_0** to predict the rating for **Movie_0** by **User_1**.



[Open in app](#)[Get started](#)

Movie_0	2	3
Movie_1	3	0
Movie_2	1	3
Movie_3	5	2

The rating for `Movie_3` by `User_1` is 2, and the rating for `Movie_0` by `User_1` is 3. If `Movie_3` and `Movie_0` are similar to `Movie_1` at the same distance, we can estimate the rating for `Movie_1` by `User_1` as 2.5. However, if `Movie_3` is considered closer to `Movie_1`, the weight for `Movie_3` should be greater than that for `Movie_0`. Therefore, the predicted rating for `Movie_1` will be closer to the rating for `Movie_3` as the picture below indicates. Using the cosine similarity as the weight, the predicted rating is 2.374.



Making a Movie Recommender

For better understanding, the dataset with 10 movies and 10 users is used here. (the numbers are randomly selected.)

df



[Open in app](#)[Get started](#)

movie_1	3	0	1	3	0	0	0	0	0	0
movie_2	0	3	0	4	0	2	0	0	0	2
movie_3	5	2	3	2	0	4	3	3	0	0
movie_4	0	5	5	0	0	0	0	0	5	4
movie_5	0	0	0	0	4	0	4	2	3	0
movie_6	4	4	0	0	4	4	3	4	0	4
movie_7	5	0	4	2	3	0	3	3	3	3
movie_8	0	3	0	0	5	5	0	4	0	0
movie_9	2	0	0	0	0	0	0	0	4	0

- 10 movies and 10 users
- 0 represents missing values.
- The percentage of users rating movies is 50% (only 50 ratings are given). In real movie datasets, this percentage becomes less than 10%.

I assume that a user did not watch a movie if the user did not rate the movie.

Calculate the Nearest Neighbors

The `NearestNeighbors()` in the `sklearn.neighbors` library can be used to calculate the distance between movies using the **cosine similarity** and find the nearest neighbors for each movie.

```
from sklearn.neighbors import NearestNeighbors

knn = NearestNeighbors(metric='cosine', algorithm='brute')
knn.fit(df.values)
distances, indices = knn.kneighbors(df.values, n_neighbors=3)
```

The number of the nearest neighbors (`n_neighbors`) is set to be 3. Since this includes the movie itself, generally it finds two nearest movies except the movie itself.

indices



[Open in app](#)[Get started](#)

```
....  
[9, 0, 7]])
```

`indices` shows the indices of the nearest neighbors for each movie. Each row corresponds to the row in the `df`. The first element in a row is the most similar (nearest) movie. Generally, it is the movie itself. The second element is the second nearest, and the third is the third nearest. For example, in the first row `[0, 7, 5]`, the nearest movie to `movie_0` is itself, the second nearest movie is `movie_7`, and the third is `movie_5`.

`distances`

```
[Out] array([[0.00000000e+00, 3.19586183e-01, 4.03404722e-01],  
[4.44089210e-16, 3.68421053e-01, 3.95436458e-01],  
[0.00000000e+00, 5.20766162e-01, 5.24329288e-01],  
....  
[1.11022302e-16, 4.22649731e-01, 4.81455027e-01]])
```

`distances` shows the distance between movies. A smaller number means the movie is closer. Each number in this array corresponds to the number in the `indices` array.

Example: Predict a Rating for a Movie by a User

Predicting a rating for a movie by a user is equivalent to calculating the weighted average of ratings for the similar movies by the user.

For practice, predict the rating for `movie_0` by `user_7`. First, find the nearest neighbors for `movie_0` using `NearestNeighbors()`.

```
# get the index for 'movie_0'  
index_for_movie = df.index.tolist().index('movie_0')  
  
# find the indices for the similar movies  
sim_movies = indices[index_for_movie].tolist()  
  
# distances between 'movie_0' and the similar movies  
movie_distances = distances[index_for_movie].tolist()
```



[Open in app](#)[Get started](#)

```
# remove 'movie_0' from the list movie_distances
movie_distances.pop(id_movie)

print('The Nearest Movies to movie_0:', sim_movies)
print('The Distance from movie_0:', movie_distances)
```

```
The Nearest Movies to movie_0: [7, 5]
The Distance from movie_0: [0.3195861825602283, 0.40340472183738674]
```

The most similar movies to `movie_0` are `movie_7` and `movie_5`, and the distances from `movie_0` are 0.3196 and 0.4034, respectively.

The formula to calculate the predicted rating is as follows:

$$R(m, u) = \{\sum_j S(m, j)R(j, u)\} / \sum_j S(m, j)$$

- $R(m, u)$: the rating for movie m by user u
- $S(m, j)$: the similarity between movie m and movie j
- $j \in J$ where J is the set of the similar movies to movie m

This formula simply implies that the predicted rating for movie m by user u is **the weighted average of ratings for the similar movies** by user u . The weight for each rating, $(S(m, k) / \sum_j S(m, j))$, becomes greater when movie m and movie k are closer. The denominator of this term makes the sum of all the weights equal 1.

Let's predict the rating for `movie_0` (👍 165 | 💬 27):

$$R(0,7)=[S(0,5)*R(5,7)+S(0,7)*R(7,7)]/[S(0,5)+S(0,7)]$$

Since the distances between `movie_0` and `movie_5` and between `movie_0` and `movie_7` are 0.4034 and 0.3196, the similarities are

- $S(0,5) = (1-0.4034)$
- $S(0,7) = (1-0.3196)$.

Because $R(5,7) = 2$ and $R(7,7) = 2$, the predicted $R(0,7)$ is 0.5228



[Open in app](#)[Get started](#)

```
watched.( n_neighbors = 3 )
```

The copy of the original dataset, `af1`, updates all the predicted ratings for `user_4`. The following code is the function to show the recommended movies for a selected user using the updated dataset.



[Open in app](#)[Get started](#)

For example, let's get 5 recommended movies for `user_4`.

```
recommend_movies('user_4', 5)
```

The list of the Movies `user_4` Has Watched

```
movie_0  
movie_5  
movie_6  
movie_7  
movie_8
```

The list of the Recommended Movies

```
1: movie_2 - predicted rating:4.0  
2: movie_3 - predicted rating:3.504943460433221  
3: movie_1 - predicted rating:3.0  
4: movie_0 - predicted rating:2.473170201820165
```



[Open in app](#)[Get started](#)

The goal of this project is to build a recommender which recommends movies for a selected user. If we enter a user name into the recommender, the recommender is supposed to return the list of recommended movies which have the highest predicted ratings. Combining the codes above, we can build the movie recommender for any selected user.

Let's recommend movies for `user_4` with `n_neighbors = 3` and the number of



[Open in app](#)[Get started](#)

The list of the Movies user_4 Has Watched

```
movie_0  
movie_5  
movie_6  
movie_7  
movie_8
```

The list of the Recommended Movies

```
1: movie_2 - predicted rating:4.0  
2: movie_3 - predicted rating:3.504943460433221  
3: movie_1 - predicted rating:3.0  
4: movie_9 - predicted rating:2.473170201830165
```

In this article, I briefly explained the basic concept of the item-based collaborative filtering and showed how to build the recommendation engine using this method.

The additional code and movie dataset for this practice can be found [here](#).

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. [Take a look.](#)

By signing up, you will create a Medium account if you don't already have one. Review our [Privacy Policy](#) for more information about our privacy practices.



Get this newsletter

